Guion de Prácticas

Programación II
1º Grado en Ingeniería de Sistemas de Telecomunicación
Universidad San Pablo - CEU
Sergio Saugar García

Abril 2026

[1]

Dune – El Imperio de Arrakis

Entrega Obligatoria: SI
Fecha límite de entrega: ***28 de Mayo de 2026***
Peso: 100 % de la nota de prácticas

Esta práctica es de **entrega obligatoria**. Tenga en cuenta que:

- La sección 4 recoge instrucciones sobre qué elementos deben ser entregados y, también, instrucciones sobre el procedimiento a seguir.
- La fecha límite para la entrega de la práctica está establecida en este enunciado. No se aceptarán entregas a partir de dicha fecha.
- Esta práctica se realizará **de manera individual**.
- Si ha realizado los ejercicios propuestos de la asignatura, valide su solución con el profesor y reutilice todo lo correspondiente con los ejercicios de Dune (diagrama *UML - Hoja 2, ejercicio 5*; jerarquía de clases - *Hoja 3, ejercicio 17*; estructuras de datos adicionales - *Hoja 3, ejercicio 12*). Simplemente **ajuste y añada** los elementos con lo definido en este enunciado.
- **Es obligatorio** el uso de un repositorio *GitHub* (en el que el profesor aparezca como colaborador), **para que la práctica sea evaluable**. En la sección 3 se establece **cómo se espera que el alumno utilice el repositorio para que la práctica sea tomada en consideración**.

1. Introducción

Esta práctica tiene como objetivo principal la consolidación de los conocimientos relacionados con el paradigma de Programación Orientada a Objetos, algoritmos de búsqueda y ordenación y

con el manejo de estructuras de datos complejas. En ese sentido, se desarrollará como aplicación un hipotético juego de simulación, utilizando el lenguaje de programación *Java* .

A continuación, en la **sección 2** se establece el **contexto y los requisitos** que debe cumplir la aplicación. La **sección 3** especifica cuestiones propias de la **evaluación de la práctica**. Posteriormente, **la sección 4 especifica qué debe incluir y cómo debe realizarse el envío de la práctica**. Finalmente, la **sección 5** establece los **mecanismos por los que se resolverán las dudas** correspondientes a esta práctica. Recuerde que las horas de ejercicios de la asignatura también estarán disponibles para la realización de este proyecto, consulta y resolución de dudas con el profesor.

2. Enunciado

Como parte de su formación en el Grado en Ingeniería de Sistemas de Telecomunicación, va a comenzar a realizar prácticas en la empresa *Complex Simulation Games* que se dedica a programar juegos para diversas plataformas online.

Recientemente, *Complex Simulation Games* ha comenzado el desarrollo de un juego de simulación tipo *Command & Conquer (C&C)*, denominado:

Dune – El Imperio de Arrakis

Esperan que el juego sea todo un revulsivo en el mercado actual y les permita conseguir financiación suficiente para ampliar el negocio y, quién sabe, quizá vender la empresa y poder retirarse!

Es la primera vez que van a desarrollar un juego para la plataforma *Java* , ya que, hasta el momento, sólo habían desarrollado juegos para móviles. Precisamente por eso, al ver sus buenas notas en *Programación II*, el jefe de proyecto decidió seleccionarle para realizar las prácticas. Pensó que no sólo podría ayudarles en las labores de programación, sino también en las de diseño del futuro software. Y, como para usted suponía un reto personal poner en práctica aquellos conceptos adquiridos en la asignatura, sin pensárselo dos veces, aceptó la propuesta.

Tras varias tormentas de ideas con los creativos de la empresa, el jefe de proyecto le hace entrega de un dosier con una explicación informal de los requisitos funcionales que debería reunir el juego. También le informa de la fecha en la que usted debería tener listo un prototipo totalmente funcional de la aplicación.

Parece que se ha embarcado en una buena aventura...

2.1. Configuración previa del equipo del alumno

Para la realización de la práctica, es necesario que:

- Tenga instalada el entorno de *Java* JDK versión 25.
- Tenga instalado el IDE *Netbeans* [2], versión 28.
  - Tenga en cuenta que **no se permitirá la utilización de versiones de *Java* diferentes a la indicada, ni de otro IDE**. El alumno deberá asegurarse que el software funcione en otro equipo diferente al suyo, que cumpla con las características señaladas.

[2] *Netbeans* . https://netbeans.apache.org/download/index.html

- Cada alumno deberá contar con **un repositorio privado en *GitHub*** único y exclusivo para este proyecto. El repositorio y el proyecto deberán cumplir las siguientes restricciones:
  - Repositorio *GitHub* con nombre: *progIIDune*.
  - El repositorio contendrá un proyecto *NetBeans* (de tipo *Java with Ant -> Java Application*), que será nombrado de la siguiente manera:

    dune-*inicialnombreprimerapellidocompleto*

    todo ello en minúsculas. Por ejemplo, si el nombre del alumno es: Sergio Saugar, el proyecto deberá llamarse ***dune-ssaugar***.
  - Una vez creado el proyecto, agregue los ficheros al repositorio y haga un *commit* inicial denominado: *"Inicio del proyecto"*.
  - **En el fichero** `README.md` **del repositorio indique el nombre completo del autor y el grado que cursa.** No se corregirá ninguna práctica que no tenga identificado al alumno en este fichero.
- El alumno deberá gestionar los permisos del repositorio *GitHub* para agregar al profesor de la asignatura como "colaborador" (el usuario *GitHub* del profesor encargado de la práctica es "ssaugar").
  - Tenga en cuenta que **el uso de un repositorio privado en *GitHub* para gestionar esta práctica es obligatorio** y servirá:
    - En primer lugar, para **poder realizar sesiones de tutorías** de una manera más eficaz. Es importante señalar que, **el profesor NI SUPERVISARÁ NI REVISARÁ** el trabajo que haya en **el repositorio fuera del ámbito de una tutoría** concertada con el alumno. El alumno es el responsable de obtener *feedback* **de manera activa**, acerca de su trabajo, a través de los medios que el profesor pone a su alcance (sección 5).
    - En segundo lugar, **el repositorio** es un **elemento clave de la evaluación**. En la evaluación, **se tendrá en cuenta todo el trabajo que quede reflejado en el repositorio**. Por ello **es obligatorio** que, **el alumno**, haga *commits* a menudo, cada vez que introduzcan algún cambio importante en el proyecto (por ejemplo, programar una clase solicitada, una característica solicitada o la corrección de un error). En cualquier caso, se aconseja realizar *commits* única y exclusivamente si el software es estable (al menos compila).
    - Mantenga **sincronizado** el repositorio remoto **constantemente**.

Todo lo relacionado con la instalación de *Java* , *Netbeans* , cuenta de *GitHub* y uso del repositorio, ya lo debió realizar el alumno como parte obligatoria de la *"Práctica 0: Configuración del sistema y manejo básico de las herramientas asociadas a la asignatura"* de la asignatura. En ese enunciado encontrará la información necesaria, si necesita refrescar dichos conocimientos.

2.2. Requisitos funcionales

Los creativos de *Complex Simulation Games* están pensando en un juego que permitirá ponerse al mando de dos de las tres grandes familias del universo *Dune* . Cada jugador intentará, por todos los medios, expandir el apellido de su *casa* por todo el planeta *Arrakis*, fundar su propio imperio, y controlar totalmente el recurso más preciado: la especia *melange*.

En este contexto, los *creativos* de *Complex Simulation Games* han pensado que *Dune – El Imperio de Arrakis* , podría ser el típico juego de gestión de recursos y conquista de territorios (¡¡¡qué creativos más originales!!!).

2.2.1. Contexto del juego

En las arenas infinitas del planeta *Arrakis*[3], las grandes *casas* nobles compiten ferozmente por el control de la *melange*: la especia más valiosa del universo. Esta sustancia no solo prolonga la vida y expande la conciencia, sino que es un combustible indispensable para la navegación espacial, permitiendo a la *Cofradía Espacial* guiar sus naves hasta los confines del universo. Por lo tanto, el control de esta especia garantiza riqueza, poder político y ventaja estratégica sobre las demás casas:

"Quien controla la especia, controla el destino de la humanidad"

Las grandes *Casas*, asociadas a antiguos linajes, establecen sus capitales en *territorios* emblemáticos de *Arrakis*, donde ejercen su influencia. Y, gracias a las *reservas de melange* que poseen dichos territorios, pueden gestionar su actividad diaria. Una de sus actividades principales es proporcionar alimento y seguridad a sus *súbditos*, antes de que el desierto los reclame. *Súbditos* necesarios para mantener la grandeza de la casa, a la que deben lealtad y obediencia.

Sin embargo, en este hostil planeta desértico, cuya superficie cubierta de dunas de arena, cordilleras y territorios rocosos dispersos dificultan la vida, las guerras y rivalidades entre casas no son los únicos problemas para la subsistencia. Las grandes *tormentas de arena* y los gigantescos moradores de las dunas, los *gusanos de arena*[4], hacen compleja la vida en *Arrakis*.

2.2.2. Estructura general de una partida

En cada *partida* hay dos jugadores. Al inicio, cada *jugador* selecciona la *casa* que desea liderar. Además, las partidas tiene un *identificador* único, de forma que el juego pueda tener almacenadas distintas partidas a punto de comenzar o ya finalizadas (a modo de histórico).

Una vez el jugador seleccione una *casa* (sección 2.2.3), ésta tendrá asociada un *territorio* (sección 2.2.4) que establecerá las condiciones iniciales del juego. Entre otras cuestiones, el *territorio* establece la cantidad inicial de *melange* (con las que la casa intentará desarrollar su imperio) y también establecerá los *súbditos* que en él residen (así como su crecimiento o decrecimiento a lo largo del juego).

Así, el objetivo del *jugador* es mantener y conquistar todos los territorios existentes en *Arrakis* para ganar en poder y extensión, creando un imperio basado en el control de la *melange*.

De esta manera, los jugadores tienen las siguientes características:

- Cada jugador tiene un *alias* dentro del juego que es su identificador único.
- Cada jugador posee una *casa*, que intentará hacer crecer y expandirse.
- Cada jugador gestionará los recursos existentes en cada uno de los *territorios* que controla su *casa*. Estos recursos, que irán modificándose (aumentando y disminuyendo) a lo largo del juego, serán utilizados para cumplir con la estrategia de conseguir formar un imperio que reine en *Arrakis*.
- Además, cada jugador cuenta con un *centro de mando* donde puede ver qué está pasando en el juego y el histórico de la a partida. Esto le permitirá observar la situación y tomar decisiones.

[3] Arrakis. https://dune.fandom.com/es/wiki/Arrakis
[4] Gusano de arena. https://dune.fandom.com/es/wiki/Gusano_de_Arena

El objetivo final de cada jugador es conquistar el mayor número posible de territorios en el planeta *Arrakis* y exiliar al resto de casas del planeta. Para ello, en cada ciclo (unidad de tiempo en el planeta *Arrakis*), el jugador podrá gestionar los recursos de sus territorios para conseguir sus objetivos.

La partida no podrá comenzar hasta que ambos jugadores no hayan elegido una *casa* y se hayan asignado súbditos al territorio asociado a la casa.

Después de esta introducción, en las siguientes secciones se describe, más detalladamente, cada uno de los elementos del juego.

2.2.3. Casas

Las casas nobles se identifican por su *apellido nobiliario*, pueden controlar varios *territorios* (sección 2.2.4) y establecen su residencia en la capital de un *territorio* emblemático. Repartidos en los territorios, las casas tienen *súbditos* (sección 2.2.5) que obedecen las órdenes de sus señores a cambio de alimento y protección. Así, las grandes casas establecidas en *Arrakis* son:

- La casa *Atreides*[5] liderada por *Duke Leto Atreides* y que se encuentra bajo la dirección de su hijo *Paul Atreides*.

- La casa *Harkonnen*[6] dirigida por el *Barón Vladimir Harkonnen*.

- La casa *Corrino*[7] que se encuentra bajo el mando del *Emperador Shaddam IV*.

2.2.4. El Mapa de *Arrakis*: Territorios y Dominios

La geografía de Arrakis es tan letal como sus intrigas políticas y las distintas *casas* gestionan territorios distribuidos a lo largo del planeta *Arrakis*.

Cada territorio posee una *extensión*, unos *silos de almacenamiento* de melange y tienen tipología concreta en función de su localización (*costero*, *árido*, *montañoso*). Esta tipología influye en la *capacidad de producción de melange* del territorio, en la *cantidad de energía* que puede

[5] Casa Atreides. https://dune.fandom.com/es/wiki/Casa_Atreides
[6] Casa Harkonnen. https://dune.fandom.com/es/wiki/Casa_Harkonnen
[7] Casa Corrino. https://dune.fandom.com/es/wiki/Casa_Corrino

producir y en la *capacidad de construcción* de asentamientos y fortificaciones. Además, en los territorios viven los distintos *súbditos* de las grandes familias, y es donde realizan sus actividades.

Históricamente, cada casa está establecida en un territorio emblemático:

- La Casa Atreides domina *Caladan*, un extenso territorio *costero* donde las planicies fértiles se mezclan con ríos serenos; su gran extensión ofrece abundante espacio para la construcción de asentamientos y fortalezas. Las crecidas de los ríos generan tierras fértiles por lo que tiene alta capacidad de producción de melange (poseyendo reservas abundantes). Las fuerzas naturales lo establecen como un enclave de generación de *energía* natural medio.
  - Ficha de *Caladan*:
    - *Extensión*: 80 Km^2
    - *Tipología*: Costero
    - *Capacidad Construcción*: Media
    - *Superficie construida* inicialmente: 20 %
    - *Capacidad de generación* de Energía: Media
    - *Energía* inicial acumulada: 100 unidades
    - *Capacidad de producción* de melange: Alta
    - Reservas de Melange inicialmente almacenadas en los *Silos*: 1500 unidades
- La Casa Harkonnen controla *Giedi Prime*, un enclave *árido* dominado por la industria; su extensión es idónea para la construcción de grandes fortificaciones y el almacenamiento de maquinaria pesada; también posee una alta capacidad de para la generación de *energía* natural. Sin embargo, debido a la persistente sequía, la producción de melange no es cuantiosa y sus reservas son limitadas.
  - Ficha de *Giedi Prime*:
    - *Extensión*: 100 Km^2
    - *Tipología*: Arido
    - *Capacidad Construcción*: Alta
    - *Superficie construida* inicialmente: 50 %
    - *Capacidad de generación* de Energía: Alta
    - *Energía* inicial acumulada: 100 unidades
    - *Capacidad de producción* de melange: Media
    - Reservas de Melange inicialmente almacenadas en los *Silos*: 1000 unidades
- Finalmente, la Casa Corrino mantiene su presencia en *Salusa Secundus*, un territorio *montañoso* y escarpado donde los riscos imposibles de escalar resguardan sus escasas instalaciones; gracias a este refugio, las *reservas de melange* acumuladas son muy grandes, aunque su capacidad de producción sea baja. Además, su altura hace que se beneficie de la generación de energía natural, esencialmente eólica.
  - Ficha de *Salusa Secundus*:
    - *Extensión*: 90 Km^2
    - *Tipología*: Montañoso
    - *Capacidad Construcción*: Baja
    - *Superficie construida* inicialmente: 10 %
    - *Capacidad de generación* de Energía: Media

- *Energía* inicial acumulada: 100 unidades
- *Capacidad de producción* de melange: BAJA
- Reservas de Melange inicialmente almacenadas en los *Silos*: 2000 unidades

Enclaves Libres Los territorios restantes de *Arrakis* se consideran disponibles para su exploración y conquista:

- Las *Arenas de Carthag*: Territorio árido, rico en especia pero costoso de defender.
  - Ficha de *Arenas de Carthag*:
    - *Extensión*: 50 Km^2
    - *Tipología*: ARIDO
    - *Capacidad Construcción*: BAJA
    - *Superficie construida* inicialmente: 5 %
    - *Capacidad de generación* de Energía: BAJA
    - *Energía* inicial acumulada: 100 unidades.
    - *Capacidad de producción* de melange: MEDIA
    - Reservas de Melange inicialmente almacenadas en los *Silos*: 150 unidades
- *Sietch Tabr*: Pequeño enclave montañoso cuyo reducido tamaño otorga una posición ventajosa en relación a su defensa.
  - Ficha de *Sietch Tabr*:
    - *Extensión*: 20 Km^2
    - *Tipología*: MONTAÑOSO
    - *Capacidad Construcción*: ALTA
    - *Superficie construida* inicialmente: 80 %
    - *Capacidad de generación* de Energía: BAJA
    - *Energía* inicial acumulada: 100 unidades
    - *Capacidad de producción* de melange: BAJA
    - Reservas de Melange inicialmente almacenadas en los *Silos*: 200 unidades
- *Arrakeen*: Territorio costero que actúa como centro logístico.
  - Ficha de *Arrakeen*:
    - *Extensión*: 80 Km^2
    - *Tipología*: COSTERO
    - *Capacidad Construcción*: MEDIA
    - *Superficie construida* inicialmente: 10 %
    - *Capacidad de generación* de Energía: MEDIA
    - *Energía* inicial acumulada: 100 unidades
    - *Capacidad de producción* de melange: MEDIA
    - Reservas de Melange inicialmente almacenadas en los *Silos*: 100 unidades

2.2.5. La Vida en el Desierto: Súbditos y sus actividades

Cada *súbdito* es un individuo que está ubicado físicamente dentro de un territorio del mapa (y, por lo tanto, rinde pleitesía a la *casa* que reina en dicho territorio), tiene unas necesidades biológicas y posee ciertas habilidades que le permiten realizar sus actividades. La eficacia de un súbdito en la actividad encomendada depende de su *energía* y *experiencia*.

Por su parte, los súbditos se organizan, según su actividad, en roles especializados:

- Los *guerreros* que constituyen la fuerza principal en combate.
- Los *exploradores* que recorren territorios no reclamados y localizan recursos estratégicos.
- Los *recolectores*, principales responsables de llenar los *silos* de los territorios con la codiciada especia que cosechan.
- Los *sabios* que ayudan a optimizar los recursos de los territorios y desarrollan nuevas tecnologías.

Salario y supervivencia de los súbditos Al final de cada ciclo, la supervivencia de los súbditos y el trabajo que realizan en las distintas actividades, tiene un precio. Las *casas* deben pagar un salario, medido en unidades de melange, a cada uno. Así:

- Los *guerreros*, pilares de la conquista y defensa de territorios, cobran 4 unidades de melange por su trabajo.
- Los *exploradores* y los *sabios* de la corte cobran 2 unidades.
- Finalmente, los *recolectores*, cuyo trabajo es fundamental para la cosecha de la especia, tienen un salario de 3 unidades.

Por una parte, en *Arrakis*, el hambre es una sentencia de muerte: si los silos del territorio, no tienen suficiente melange para cubrir su salario, aquellos súbditos que no lo reciban, morirán. Por otra parte, aquellos que reciban su salario no solo sobrevivirán, sino que recuperarán 2 puntos de energía y ganarán 5 unidades de experiencia.

Además, aquellos afortunados que habiten territorios con más de un 60 % de la superficie construída, disfrutarán de unas comodidades que les otorgará 2 puntos extra de energía por cada ciclo.

Nacimiento de los súbditos En todos los territorios que están bajo control de una *casa*, la demografía natural hace que nazcan entre 1 y 3 súbditos, cada 5 ciclos. La actividad a la que se dedicarán es una incógnita y depende del azar. Lo único que se sabe es que el 30 % de los súbditos prefieren las labores de *recolección*, el 40 % prefieren actividades guerreras, el 20 % les gustan las tareas de exploración y el 10 % son estudiosos que llegarán a ser sabios.

Traslado de los súbditos entre territorios Las *casas* pueden reorganizar a sus súbditos entre sus territorios, de forma que puedan realizar sus actividades en otros lugares. Sin embargo, el viaje entre los distintos territorios afecta a la energía de los súbditos, que disminuye en 20 unidades cada vez que se trasladan.

La cosecha de melange La tarea principal de los *recolectores* es cosechar la melange que se produce en un territorio. De esta manera, en cada ciclo, se recolecta cierta cantidad de especia que irá a parar a los *silos* del territorio. La cantidad de especia recolectada en cada ciclo, es la siguiente:

$$cantidad_{especia} = \alpha * \frac{num_{recolectores} * \beta}{100}$$

$$\text{siendo } \alpha \begin{cases} 2 & \text{si capacidad de producción: BAJA} \\ 5 & \text{si capacidad de producción: MEDIA} \\ 10 & \text{si capacidad de producción: ALTA} \end{cases}$$

y siendo β la media de la experiencia que tienen los recolectores que están en dicho territorio.

2.2.6. El Flujo Energético: Logística e Ingeniería

La energía es el motor de la tecnología en *Arrakis* y es utilizada para la construcción de fortificaciones y asentamientos para los súbditos. Así, elevar el nivel de construcción de un territorio depende de distintos factores: la capacidad de construcción del mismo, la cantidad de territorios ya construidos, la energía asociada a la construcción y, también, la intervención de *sabios* que optimicen los recursos y ayuden a aplicar la tecnología.

Se podría decir que, para aumentar el nivel de construcción de un territorio en un 1 %, la energía necesaria sería la siguiente:

$$energia_{requerida} = \alpha * \beta * \frac{territorio_{extension}}{superficie_{construida}}$$

$$\text{siendo } \alpha \begin{cases} 3 & \text{si capacidad de construcción: BAJA} \\ 1 & \text{si capacidad de construcción: MEDIA} \\ \frac{1}{2} & \text{si capacidad de construcción: ALTA} \end{cases}$$

y

$$\text{siendo } \beta \begin{cases} 3 & \text{si } 0 \leq superficie_{construida} < 33 \\ 2 & \text{si } 33 \leq superficie_{construida} < 80 \\ 1 & \text{si } superficie_{construida} \geq 80 \end{cases}$$

Si el territorio tiene un conjunto de *sabios* residiendo en él, la energía requerida se verá reducida en relación a la media de experiencia que tengan todos esos sabios. Para ello, se calculará la media, y la $energia_{requerida}$ calculada anteriormente será multiplicada por el factor $\frac{media_{experiencia}}{4}$.

Al final de un ciclo, la energía que produce un territorio se calcula de la siguiente manera:

$$energia_{generada} = \alpha * \frac{superficie_{construida} * extension_{territorio}}{100}$$

$$\text{siendo } \alpha \begin{cases} \frac{1}{2} & \text{si capacidad de generación: BAJA} \\ 1 & \text{capacidad de generación:: MEDIA} \\ 2 & \text{capacidad de generación: ALTA} \end{cases}$$

2.2.7. La Guerra por la Especia: Conflictos y Conquista

Las *casas* en su búsqueda de poder pueden conquistar nuevos territorios o atacar territorios pertenecientes a otras familias. Para ello, ordenan a sus súbditos moverse a estos territorios, lo que denominan enviar una expedición.

- Si se desea conquistar un territorio libre, se envía un conjunto de exploradores al territorio, formando una expedición de reconocimiento.
- Si se desea conquistar un territorio ocupado, se envía un conjunto de guerreros al territorio, formando una expedición de conquista.
- Si se desea ayudar a un territorio que está intentando ser reconocido o conquistado, se puede enviar un conjunto de exploradores o guerreros.

Todas las expediciones a otro territorio provocan un conflicto que debe resolverse, y esa resolución depende de quién lo esté ocupando en dicho momento:

- **Explorador vs. Territorio vacío**: Cuando una casa envía exploradores a un territorio vacío, la conquista del mismo dependerá de si son capaces o no de estabilizar el territorio. Para saber si lo podrán estabilizar:
  - Se calcula la media de la experiencia de los exploradores y la media de la energía que les queda.
  - Se multiplican ambas medias y se denomina factor de estabilización.
  - Se saca un número aleatorio entre cero y tres veces el factor de estabilización.
    - Si el número aleatorio es inferior al factor de estabilización, el territorio es conquistado por la *casa* a la que pertenecen los exploradores que pasará a controlar todas las riquezas y súbditos presentes en el mismo
    - Si el número aleatorio es igual o superior al factor de estabilización, los exploradores fallan en su hazaña y mueren intentando finalizar su misión.
- **Explorador vs. Explorador**: Si ambas *casas* envían exploradores a un territorio libre, no hay combate. El territorio quedará en disputa hasta que una de las dos decida enviar guerreros para conquistar el territorio o retirar a sus exploradores.
- **Exploradores vs. Guerreros**: Si una *casa* envía exploradores a un territorio donde el rival tiene (o envía) guerreros, los exploradores son eliminados automáticamente del territorio y el territorio es asignado a la *casa* de los guerreros, que pasará a controlar todas las riquezas y súbditos presentes en el mismo.
- **Guerreros vs. Guerreros**: Si un jugador envía a guerreros a un territorio donde el rival tiene (o envía) guerreros, se establece un combate. En este combate:
  - Se calcula el poder de defensa que tienen los guerreros defensores (los guerreros que son súbditos de la *casa* regente en dicho territorio). Ese poder de defensa depende de la experiencia media de todos los guerreros, incrementada por los sitios donde pueden resguardarse.

$$potencia_{defensa} = media_{experiencia} * \frac{100}{100 - superficie_{construida}}$$

- Se calcula el poder de ataque que tienen los guerreros invasores. Ese poder depende de la experiencia media de los guerreros invasores y se ve potenciado por la energía que tengan y la tipología del territorio (que puede contribuir a parar o facilitar el ataque).

$$potencia_{ataque} = media_{experiencia} * \frac{100}{100 - media_{energia}} * \alpha$$

$$\text{siendo } \alpha \begin{cases} 3 & \text{si tipología: COSTERO} \\ 2 & \text{si tipología: ARIDO} \\ 1 & \text{si tipología: MONTAÑOSO} \end{cases}$$

- El bando con mayor potencia se queda con el territorio y los guerreros del bando contrario, son aniquilados. Por lo tanto, el territorio es asignado a la *casa* de los guerreros triunfales, que pasará a controlar todas las riquezas y súbditos presentes en el mismo.

2.2.8. Los Gusanos de las Arenas y las Tormentas

Arrakis es un planeta traidor que castiga a los imprudentes.

- La extracción intensiva de melange genera vibraciones que atraen a los *Gusanos de Arena*. En aquellos territorios cuya cantidad de especia cosechada en un ciclo supere el 5 % de las reservas acumuladas en el territorio, la probabilidad de que aparezca un gusano es de $0{,}5 * numero_{recolectores}$. En caso de que aparezca, el gusano devorará toda la producción de melange de ese ciclo y aniquilará a la mitad de los recolectores que estén en el territorio.

- La Tormenta de Coriolis siempre acecha, a cualquier territorio, con un 15 % de probabilidad, siendo capaz de reducir a la mitad la energía de todos los súbditos del territorio. Solo aquellos territorios que tienen más de un 60 % de superficie construida y energía suficiente para alimentar sus escudos de viento (150 unidades mínimo), podrán ignorar la furia del viento.

2.2.9. Configuración de la partida

De esta manera, la secuencia de pasos que hay que realizar para configurar la partida, y poder comenzar el juego, es la siguiente:

1. Cada jugador seleccionará la *casa* que desea controlar. Este hecho establecerá el *territorio* inicial del que dispondrá.

2. Se generará una población de 20 súbditos para cada uno de los territorios iniciales de ambas casas. Esta generación será aleatoria y cumplirá con lo establecido en la sección 2.2.5 (nacimiento de los súbditos).

3. A partir de ahí, de cada jugador podrá seleccionar una acción:

   a) Trasladar a unos súbditos concretos de un territorio que posee a otro territorio.

   b) Incrementar, en cierto porcentaje, el nivel de construcción de un territorio que posea.

   c) Una vez realizada la acción de cada jugador, comienza la simulación de la partida.

2.2.10. Simulador de la partida

En esta sección se describirá cómo será la simulación de la partida. En este caso, se propone una aproximación automática, donde, en cada ronda se simulará el paso de un ciclo temporal de *Arrakis* y será el ordenador quién calculará lo que va sucediendo. Tras cada ronda, el jugador tendrá, de nuevo, la oportunidad de ir gestionando los recursos pertenecientes a su *casa*.

El simulador recibirá la partida configurada y, en cada ronda de simulación, hará lo siguiente:

1. En primer lugar, se comprobará el resultado de las distintas *expediciones* que hayan tenido lugar.
2. En segundo lugar, se actualizará toda la información relativa a los *súbditos* que quedan en los diferentes territorios.
   - Por cada súbdito, se procederá al pago de su salario (o se declarará su muerte) y se actualizarán su *energía* y *experiencia*.
   - Si el ciclo es un ciclo de nacimientos (cada 5 ciclos), se actualizarán los nuevos súbditos de manera adecuada.
3. En tercer lugar, se actualizará toda la información relativa a los *territorios*.
   - Se actualizará la energía que se ha generado en este ciclo.
   - Se actualizará la cantidad de especia que ha sido recolectada en este ciclo.
4. Por último, se comprobará si los gusanos de las arenas y las tormentas afectan a los distintos territorios (y se aplicarán sus consecuencias).

Antes del inicio otra nueva simulación, se incrementará el ciclo y, primero a un jugador y después al otro, se les dará la opción de ejecutar **UNA de las posibles acciones del juego** (sin incluir la consulta del *centro de mandos*, que podrán ejecutarla, durante su turno, las veces necesarias para tomar una decisión y ejecutar una acción):

- Consultar su *centro de mandos* para analizar cuál es el estado de la partida y los distintos resultados de la simulación (estado de todos los terrenos, súbditos, etc..).
- Trasladar a unos súbditos concretos de un territorio que posee a otro territorio. Sólo es factible mover súbditos concretos que pertenezcan a la *casa* y siempre que tengan energía suficiente (se descontará la energía correspondiente al movimiento). Se podrán elegir tantos súbditos como el jugador desee, pero el movimiento se producirá desde el mismo territorio a otro único territorio destino.
- Incrementar, en cierto porcentaje, el nivel de construcción de un territorio que posea. Sólo si la cantidad de energía del territorio lo permite (se descontará la cantidad correspondiente a la construcción realizada).
- Pasa turno y no realiza ninguna acción en este ciclo.

Una vez que ambos jugadores han realizado **LA ACCIÓN** deseada, se ejecutará de nuevo la simulación y se incrementará el número de ciclo.

En este caso, la partida sólo termina cuando un jugador ha conquistado todos los territorios disponibles en *Arrakis*.

2.2.11. Centro de mandos

El centro de mandos es un dispositivo imprescindible para que cada jugador pueda observar el estado del juego. De esta manera, el centro de mandos de cada jugador deberá permitir:

- Mostrar por pantalla todos los territorios de *Arrakis* y sus características. Los territorios se mostrarán ordenados de manera decreciente por la cantidad de melange almacenada en sus *silos*.
- Mostrar por pantalla todas las acciones que han ocurrido hasta el momento: tanto las acciones que ha realizado cada uno de los dos jugadores, como las surgidas como consecuencia de la simulación. Estas acciones se mostrarán ordenadas, de manera creciente, por la fecha en la que ocurrieron.

Adicionalmente, el cuadro de mandos encapsulará todas las funciones de salida que el jugador requiera y que el diseñador crea necesarias (por ejemplo: mostrar los datos concretos de un terreno, mostrar todos los terrenos de una *casa* que esté participando en el juego, mostrar a todos los súbditos de un tipo concreto, etcétera).

2.2.12. Gestión de errores y excepciones

Además del cumplimiento de la funcionalidad establecida, se requiere que el software incluya todos los chequeos (y tratamiento de excepciones) necesarios para asegurar que los datos procesados en la aplicación sean consistentes.

2.2.13. Ampliaciones

Se anima al alumno a que, una vez que realice la implementación de estos requisitos iniciales, amplie la funcionalidad del juego. En ese sentido, se propone:

- Mejoras en la jugabilidad. Como ejemplos podríamos señalar:
  - Mejoras en el sistema de cálculos proporcionados por el enunciado: que cada súbdito tenga un atributo de eficacia que permita calcular (de forma más efectiva que la media) su rendimiento en una tarea, que sean más realistas las fórmulas de cálculo automático (por ejemplo: energía generada),...
  - Inclusión de nuevos elementos del juego: territorios, súbditos, ...
- El alumno podrá proponer cualquier otra mejora al profesor que la valorará, pudiendo aceptarla para su inclusión en la evaluación.

Estas ampliaciones permitirán al alumno obtener hasta dos puntos extra sobre la nota original.

2.3. Operaciones de entrada/salida de datos

Se desea que todos los datos del juego se almacenen en un fichero. Así, el juego guardará los datos que maneja: si se lo pide el usuario **y, siempre, al salir de la aplicación**. De la misma manera, el juego podrá cargar todos los datos de sesiones anteriores: si se lo pide el usuario **y, siempre, al arrancar la aplicación**.

2.4. Interfaz textual/gráfica

Se requiere que la aplicación tenga una interfaz que permita a dos jugadores:

- Configurar la situación inicial de la partida (selección de casa e inicialización de los súbditos del terreno).
- Desarrollar el juego en base a la ejecución de la dinámica de rondas establecida.

Por último, la interfaz deberá dar la posibilidad de guardar datos de partidas justo antes de iniciar una nueva ronda. De esta manera, la partida podrá ser guardada y cargada posteriormente para proseguir desde el mismo punto en que se dejó.

2.5. Consejos

- Varios puntos del enunciado están abiertos, tome sus decisiones de diseño y coméntelas con el profesor para que las valide.
- Organice y estructure el código usando herencia, interfaces, polimorfismo...
- Establezca la visibilidad de atributos y métodos para asegurar la encapsulación de los datos.
- Reutilice todo el código que pueda (constructores del padre, código de métodos heredados, etcétera).
- Estructure su código en pequeños métodos que realicen funciones concretas.
- Recuerde que, en POO, es fundamental que cada clase sea responsable de la modificación de su estado. De esta manera, cada clase deberá ser responsable de modificar sus atributos. Evite el uso de *getters* y *setters* para modificar los atributos propios de cada clase desde otra, de manera externa.
- Organize el código de la práctica en varios paquetes: las clases que representan datos y lógica de la aplicación en el paquete *modelo*, la clase almacén de datos en el paquete *data*, las clases que trabajan con entrada/salida a disco en el paquete *io*, las clases de la interfaz de usuario en el paquete *gui*, etcétera.

3. Evaluación

Para superar la práctica, además del software entregado, **el alumno realizará una defensa** (respondiendo a preguntas formuladas por el profesor sobre funcionamiento, cuestiones de diseño, partes del código, etcétera). Si el alumno **no superase la defensa de la práctica, la nota final de la práctica será de cero puntos**. Tal y como se establece en la guía docente: "*Los alumnos* ***pueden ser convocados*** *a realizar la defensa de cualquiera de las prácticas que entreguen,* ***en cualquier momento hasta el fin de la convocatoria ordinaria****. La* ***defensa*** *se podrá realizar* ***tanto oralmente como de manera escrita***".

Además de la defensa, la práctica se evaluará en relación a:

- **Organización y estructuración del código** (utilización de conceptos y patrones de programación orientada a objetos: herencia, polimorfismo, encapsulación, reutilización, utilización correcta de estructuras de control, ...).
- Utilización de **estructuras de datos adecuadas** y utilización de **técnicas de ordenación y búsqueda**, ... El uso de todas ellas deberá ser analizado y justificado **en la memoria descriptiva.**

- **Uso de *Javadoc*** para documentar las clases y métodos.
- Funcionamiento ajustado a los requisitos establecidos (**incluyendo, además de chequeos de datos, integridad de la información, gestión de excepciones...**).
- Si se implementan **algunas de las ampliaciones** recogidas en el apartado 2.2.13 o alguna **ampliación acordada con el profesor**, se podrán conseguir hasta dos puntos extra.
- **Claridad del código y adecuación a las normas de estilo** (correcto nombrado de clases, métodos y variables, comentarios internos, indentación del código,. . .).
- **Trabajo continuado y detallado que el alumno haya realizado a lo largo de la práctica**. Para verificar este apartado se revisarán todos los commits realizados en el repositorio de *GitHub* (**que es obligatorio utilizar para el seguimiento de la misma**). Se tendrá en cuenta: **la calidad de los commit**s y la **realización de manera incremental y progresiva** de la práctica. Es decir, el trabajo en el repositorio debe **reflejar** que el alumno ha estado trabajando en la práctica de manera continuada y que todo el trabajo ha sido realizado por él mismo.

Se advierte al alumno de que, como parte de la evaluación, se utilizará una **herramienta antiplagio** de código. En caso de copia total o parcial del código de la práctica, **se aplicarán las medidas de sanción correspondientes**.

3.1. Utilización del repositorio *GitHub*

Para que la práctica sea evaluable es **obligatoria** la utilización de un repositorio *GitHub* . Allí, cada alumno **registrará toda la actividad de desarrollo de la práctica**.

De esta manera, se espera que el alumno vaya realizando *commits* de su trabajo cada vez que implemente algún trabajo significativo sobre el software (creación de una nueva clase, un nuevo método o funcionalidad, modificación de los atributos de una clase,. . .). Es decir, **el alumno debe realizar *commits* de manera regular** y con un **propósito preciso** (se debe evitar hacer *commits* por el mero hecho de incrementar la cantidad de los mismos; deben hacerse con el objetivo de registrar un nuevo incremento válido en el software). Se recomienda que se asegure de que el proyecto compila antes de hacer un *commit*.

Es responsabilidad del alumno realizar los *commits* de manera adecuada, **de forma que se pueda evaluar que su trabajo se ha realizado de manera incremental**. También es responsabilidad suya **comprobar** que los *commits* se están **registrando correctamente en el repositorio remoto**.

Se advierte que las siguientes situaciones: hacer un único (o unos pocos) *commit*, hacer *commits* sin un propósito razonable (sin incremento de software asociado) o realizar una gestión de los *commits* **que no permitan comprobar que el alumno ha trabajado constantemente y de manera incremental** en el proyecto, **pueden derivar en el suspenso de la práctica.**

Por lo tanto, **antes de comenzar a codificar la práctica**, y tal y como se exponía en la sección 2.1:

- Genere un **repositorio privado** en *GitHub* , clónelo en su disco duro y genere un proyecto *Netbeans* en dicha carpeta, siguiendo las instrucciones que se encuentran en la sección 2.1.

- **Los nombres del repositorio y del proyecto** deberán ajustarse a lo establecido al inicio de este enunciado en la sección 2.1.
- Una vez configurado correctamente, haga un primer *commit* añadiendo el proyecto y subiéndolo al repositorio remoto, tal y como indica la sección 2.1.
- Recuerde que, posteriormente, los *commits, pull y push* podrá realizarlos, si lo desea, directamente desde *Netbeans*[8].
- No se olvide de agregar al profesor como colaborador de su repositorio, tal y como indica la sección 2.1.

Si tiene dudas acerca de cómo crear o manejar un repositorio de *GitHub* **revise el enunciado de la práctica 0.** Si sigue con dudas, **concierte una tutoría con el profesor de la asignatura, ANTES DE COMENZAR A PROGRAMAR EL SOFTWARE**.

4. Entrega de la práctica

Para aceptar la entrega de la práctica es **condición necesaria** que el código entregado **compile correctamente**. Si el código presentase errores de compilación, la entrega sería rechazada, no evaluada y calificada con cero puntos.

4.1. Entrega final - *28 de Mayo de 2026*

- **Entrega obligatoria**.
- La evaluación de esta entrega tendrá un **peso del 100 %** de la nota de prácticas del alumno, respecto al método de evaluación que esté siguiendo.
- Deberá entregar la implementación íntegra de los requisitos establecidos en el enunciado de la práctica.
- Se entregará también una **memoria descriptiva** del trabajo realizado. Esta memoria tendrá un **peso del 10 %** dentro de la nota de la práctica.

Como resultado de esta entrega final se deberá enviar un fichero *zip* con la carpeta completa que contenga el proyecto software *Netbeans* correspondiente a la práctica (asegúrese de que incluye todo el código fuente).

Además, en su repositorio de *GitHub* deberá realizar un *commit* denominado *"Dune Entrega Final"* indicando así, el punto temporal exacto de la entrega.

El envío de la memoria y el fichero se realizará siguiendo estas instrucciones:

- La memoria se ajustará a la siguiente estructura:
  - Portada con el nombre del alumno, su titulación, su usuario de *GitHub* y la dirección *URL* del repositorio.
  - Sección "Análisis y descripción de la aplicación":
    - Se incluirá, **obligatoriamente**, un **Diagrama de Clases** *UML* completo.

[8] Using Git in Apache Netbeans. https://netbeans.apache.org/kb/docs/ide/git.html

    - Este análisis y descripción contendrá las siguientes subsecciones:
      - Subsección "Organización y Estructura". Donde debe explicar cómo se han organizado y estructurado las clases y cuál es la responsabilidad de cada una (para ello, básese en el diagrama *UML*).
      - Subsección "Decisiones de Diseño". Donde debe explicar qué decisiones de diseño se han tomado.
      - Subsección "Comprobaciones de integridad de la información y Excepciones". Donde debe explicar qué comprobaciones de integridad (y excepciones) se han implementado.
      - Subsección "Uso de Estructuras de Datos". Donde debe explicar qué estructuras de datos ha utilizado y por qué ha elegido dichas estructuras y no otras.
      - Subsección "Técnicas de Ordenación y Búsqueda". Donde debe explicar qué técnicas de ordenación y búsqueda ha utilizado y por qué ha elegido dichas técnicas y no otras.
  - Sección "Listado de fallos conocidos y funcionalidades no implementadas". Donde explicará los fallos que tiene su aplicación y, en su caso, el conjunto de **funcionalidades definidas en el enunciado que no ha podido implementar**.
  - Sección "Conclusiones". Que incluirá, **obligatoriamente**, una valoración del tiempo dedicado a la práctica.

- La memoria y el fichero se enviarán a través de la actividad que será creada a tal efecto en el *Campus Virtual* de la asignatura. **No se aceptará ninguna entrega por otros medios (correo electrónico...)**.

- Recuerde que la **entrega de la práctica deberá realizarse dentro del período de tiempo indicado**, en caso contrario, se evaluará con **cero puntos**.

5. Tutorías

Se recomienda que el alumno asista a reuniones periódicas con el profesor para asegurarse de que está estructurando y organizando el código de una manera correcta. Estas reuniones, consultas y dudas que surjan en el desarrollo de esta práctica, se resolverán **prioritariamente** en las clases dedicadas a la resolución de problemas y ejercicios. Adicionalmente, también se resolverán durante las horas de tutorías establecidas. Para ello, es necesario concertar, previamente, una cita con el profesor a través del correo electrónico.

6. Referencias

Los nombres, personajes, sitios e imágenes que aparecen en este enunciado están registrados por sus respectivos dueños. La información que ha servido para confeccionar este enunciado, ha sido extraída de los siguientes sitios:

1. *Logo Dune*
   Por *Frank Herbert - Dune by Frank Herbert*, Dominio público, https://commons.wikimedia.org/w/index.php?curid=124064875

2. *Dune Wiki Fandom*
   https://dune.fandom.com/es/wiki/Dune_Wiki